

you can use the standard library function `List.fill(n)(x)` to make a list with `x` repeated `n` times.

```
def sequence[A](fs: List[Rand[A]]): Rand[List[A]]
```

6.4.2 Nesting state actions

A pattern is beginning to emerge: we're progressing toward implementations that don't explicitly mention or pass along the RNG value. The `map` and `map2` combinators allowed us to implement, in a rather succinct and elegant way, functions that were otherwise tedious and error-prone to write. But there are some functions that we can't very well write in terms of `map` and `map2`.

One such function is `nonNegativeLessThan`, which generates an integer between 0 (inclusive) and `n` (exclusive):

```
def nonNegativeLessThan(n: Int): Rand[Int]
```

A first stab at an implementation might be to generate a non-negative integer modulo `n`:

```
def nonNegativeLessThan(n: Int): Rand[Int] =
  map(nonNegativeInt) { _ % n }
```

This will certainly generate a number in the range, but it'll be skewed because `Int.MaxValue` may not be exactly divisible by `n`. So numbers that are less than the remainder of that division will come up more frequently. When `nonNegativeInt` generates numbers higher than the largest multiple of `n` that fits in a 32-bit integer, we should *retry* the generator and hope to get a smaller number. We might attempt this:

```
def nonNegativeLessThan(n: Int): Rand[Int] =
  map(nonNegativeInt) { i =>
    val mod = i % n
    if (i + (n-1) - mod >= 0) mod else nonNegativeLessThan(n)(???)
  }
```

Retry recursively if the `Int` we got is higher than the largest multiple of `n` that fits in a 32-bit `Int`.



This is moving in the right direction, but `nonNegativeLessThan(n)` has the wrong type to be used right there. Remember, it should return a `Rand[Int]` which *is a function* that expects an RNG! But we don't have one right there. What we would like is to chain things together so that the RNG that's returned by `nonNegativeInt` is passed along to the recursive call to `nonNegativeLessThan`. We could pass it along explicitly instead of using `map`, like this:

```
def nonNegativeLessThan(n: Int): Rand[Int] = { rng =>
  val (i, rng2) = nonNegativeInt(rng)
  val mod = i % n
  if (i + (n-1) - mod >= 0)
    (mod, rng2)
  else nonNegativeLessThan(n)(rng)
}
```